

# Edge cases (where the $\frac{3}{4}$ -word rule breaks) — Hints

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (why it happens), **Advanced** (what to actually do about it in real code). Read Basic first even if you already know the rule — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Why the rule breaks, and where](#)
- [Hint 2 — Worked examples, and the real numbers](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — Why the rule breaks, and where

---

### Basic Version

The "1 token  $\approx \frac{3}{4}$  of a word" rule was worked out from ordinary English prose. It's not a law of nature — it's just a description of how the tokenizer happens to behave on the kind of text it saw a lot of during training. Anything that isn't ordinary English prose is fair game to break the rule badly.

Pick one snippet of Python code, and one sentence written in a language other than English, and just guess their token counts using the rule before you look anything up.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

The tokenizer's vocabulary is built by a statistical process (byte-pair encoding, roughly: repeatedly merging the most frequent adjacent character pairs seen in training) run over a training corpus that's mostly English prose. Whatever pattern shows up often in that corpus earns a short, efficient token. Whatever doesn't — rare words, punctuation-heavy code syntax, non-Latin scripts — gets broken down into smaller, less efficient pieces, because the exact chunk wasn't common enough in training to deserve its own single token.

For code: count the punctuation characters separately from the words — parentheses, quotes, colons, and brackets each often become their own token, which is why code "looks short" but tokenizes long.

For non-English text: don't try to guess from an English rule of thumb at all here — just note that the same *meaning*, in a language with less training data behind it, usually costs noticeably more tokens than the English equivalent.

Write your guesses for the code snippet and the non-English sentence, in tokens, before checking them.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

The deeper pattern connecting both exceptions (code, non-English text) is the same: **token efficiency tracks training-data frequency, not human-perceived complexity**. A short Urdu sentence isn't linguistically more complex than its English translation — it just has far less representation in the corpus the tokenizer's vocabulary was built from, so the tokenizer has fewer efficient whole-chunk tokens reserved for it and falls back to smaller, less efficient pieces.

This has a real consequence beyond trivia: any two features that feel equally simple to a developer (a support bot in English vs. the same bot in Urdu; a code-formatting feature vs. a prose-formatting one) can have very different token costs and context-budget footprints, purely because of how the training corpus happened to be weighted — not because one task is harder than the other by nature.

Given this, predict: would a JSON-heavy system prompt (lots of `{`, `}`, `"`, `:`) tokenize better or worse than the equivalent information written as plain English sentences? Write your prediction, then check it against a real tokenizer.

**Difference between Basic, Intermediate, and Advanced:** Basic names that the rule breaks outside plain English. Intermediate explains the training-data mechanism behind why. Advanced generalizes that mechanism into a rule you can apply to *any* new content type you haven't tested yet — token efficiency tracks training-data frequency, not perceived complexity — and asks you to predict a genuinely new case (JSON) using that rule before verifying it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — Worked examples, and the real numbers

---

### Basic Version

```
code snippet: "print('hello world')"  
word-count guess: about 3 tokens (2 words + quotes)  
actual: usually 5-7 tokens – each punctuation mark counts separately  
  
urdu sentence: "میں آج بازار جا رہا ہوں" (I am going to the market today)  
word-count guess: about 7 tokens (5 words)  
actual: usually much higher – often 2-3x the English equivalent
```

The point isn't to memorize a multiplier — it's to know when to stop trusting the quick word-count rule entirely and just measure.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

```
code: "print('hello world')"  
naive word-count estimate: ~3 tokens  
actual (tokenizer tool): ~5-7 tokens  
gap explained by: print, (, ', hello, world, ', ) each frequently tokenizing  
separately or near-separately – punctuation isn't "free"  
  
non-English: "میں آج بازار جا رہا ہوں"  
English equivalent: "I am going to the market today" (~7 tokens)  
actual for the Urdu sentence: often 15-20+ tokens  
gap explained by: far less Urdu text in the tokenizer's training data means  
the vocabulary has few or no efficient multi-character  
chunks for this script – most of it falls back to small,  
inefficient sub-word or byte-level pieces
```

Run your own two examples through the real tokenizer tool now and record the actual counts. Write down which parts of your own future projects fall into these exception categories: system prompts full of JSON schema examples, a multilingual support bot, or a feature that echoes back user-pasted code.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

Turn this into something you'd actually keep in a real codebase: a small helper that measures instead of estimates, so nobody on the team has to remember which content types are exceptions.

```
import tiktoken

def real_token_count(text: str, model: str = "gpt-4o") -> int:
    encoding = tiktoken.encoding_for_model(model)
    return len(encoding.encode(text))
```

Call this on anything that isn't plain English prose before it ships — a system prompt template, a set of few-shot examples, a multilingual UI string — instead of eyeballing a word count. Then think one step further: if you're budgeting cost for a feature that serves multiple languages, the right approach isn't one global average multiplier, it's measuring the *actual* per-language average from real traffic, since the gap varies a lot by language (some non-Latin scripts are far worse than others).

**Difference between Basic, Intermediate, and Advanced:** Basic and Intermediate get you accurate numbers for the 2 examples in front of you, once. Advanced turns that one-off check into a reusable measurement habit — a function you actually call in code, and a per-language measurement strategy instead of a single guessed multiplier — which is what keeps a real multilingual or code-heavy feature's cost estimate honest as it grows past the 2 examples you happened to test by hand.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)